

SmartJourney

Test Plan Report

AI-Powered Smart Tourism Assistant Platform with Personalized Trip Planning

Date: 2026-09-27

Group 08

230247T – HEWAWASAM H.H.S.T.T.

230279R - JAYASINGHE E.A.T.D.

Contents

Contents	1
1. Evaluation Mission and Test Motivation	2
2. Target Test Items	2
Backend (NestJS)	2
AI backend (FastAPI)	2
Frontend (Next.js)	2
Infrastructure	3
Out of scope	3
3. Test Approach	3
3.1 Testing Techniques and Types	3
3.1.1 Data and Database Integrity Testing	3
3.1.2 Function Testing	5
3.1.3 User Interface Testing	7
3.1.4 Performance Profiling	9
3.1.5 Load Testing	10
3.1.6 Security and Access Control Testing	11
3.1.7 Failover and Recovery Testing	12
3.1.8 Configuration Testing	14
4. Deliverables	17
4.1 Test Evaluation Summary (as actually run, 2026-09-27)	17
4.2 Reporting on Test Coverage	19
4.3 Security and Access Control Test Cases	19
4.3.1 Security testing — authentication	19
4.3.2 Access control testing — authorization	20
5. Risks, Dependencies, Assumptions, and Constraints	23
6. References	23
Appendix A — Manual Test Log Template	24
Appendix B — Screenshot Checklist	25

1. Evaluation Mission and Test Motivation

This Master Test Plan covers SmartJourney, a Sri Lanka trip-planning system built from three independently deployable services: a Next.js web frontend, a NestJS API backend, and a FastAPI/LangGraph AI backend that plans itineraries. A fourth planned client, a Flutter mobile app, has not been started and is out of scope for this plan.

Unlike a single MVC application, there is no one place where "every Model, View and Controller" lives — the mission here is to test every service boundary: browser to NestJS, NestJS to FastAPI, FastAPI to the LLM and external data APIs, and NestJS to the Postgres/PostGIS database. A defect at any one of those boundaries can be invisible to tests that only exercise the service on either side of it.

The main objectives of the testing process are:

- Find bugs before deployment, with special attention to data isolation - a traveler's itineraries, expenses, and chat history must never be readable by another traveler.
- Find design/implementation problems that threaten quality. This plan's own construction found several real defects (see Section 4.1) among them a Prisma filter that silently dropped a date bound, and an orchestrator crash on a failed follow-up turn.
- Identify risks, especially around the free-tier external APIs the itinerary planner depends on (Section 5).
- Meet the quality standard appropriate to an LLM-driven core: Since the AI backend produces non-deterministic outputs, AI-generated results shall not be evaluated based on exact textual similarity. Instead, correctness shall be evaluated using defined criteria including schema validity, budget compliance, policy compliance, contextual relevance, and successful execution of the fallback mechanism when the LLM or external AI services are unavailable.
- Meet functional and non-functional requirements, verified against the golden-scenario suite already maintained for the AI backend.
- Report honestly. Every number in this document was produced by actually running the relevant test suite, not estimated - and where something could not be run in this environment (a scenario needing a live signed-in session), that is stated rather than assumed passing.

2. Target Test Items

As this system is three cooperating web services rather than one MVC application, the test items below are grouped by service instead of by architectural layer.

Backend (NestJS)

8 modules (auth, users, trips, budget, chat, explore, admin, app), about 50 HTTP routes, the global JwtAuthGuard/RolesGuard chain, the Prisma/PostGIS data layer, and the raw-SQL migration runner.

AI backend (FastAPI)

The planner and recommendation agents; core orchestration (orchestrator, react, scoring, clustering, budget, itinerary, fallback, output validator, follow-up); tools for weather, disaster, geocoding, routing, calendar, and database access; and data connectors for OSM, Booking.com, Ticketmaster, and Wikidata.

Frontend (Next.js)

Route middleware (authentication/role gating), the Keycloak token-refresh flow, 8 page routes, the Zustand trip store, and the chat/budget/explore/admin components.

Infrastructure

The Docker Compose stack (PostGIS, Redis, Keycloak, Mailpit), the Keycloak realm import, and cross-service network isolation - the AI backend must never be reachable from outside the Docker network.

Out of scope

The Flutter mobile app (not yet started). Subscription/billing (not built - the analytics service explicitly reports this as unavailable rather than inventing a figure).

3. Test Approach

The testing strategy focuses on meaningful coverage rather than simply maximizing the number of tests. This round added 87 new tests in total (38 backend unit tests across 8 modules, 6 backend end-to-end tests, 18 frontend unit tests, and 4 Playwright scenarios) rather than one test per route or component - see Section 4 for what each one checks and why.

Technique	Primary tool(s)
Data & DB Integrity	Prisma schema + hand-written Prisma fakes in unit tests; psql/Prisma Studio against the PostGIS container
Function Testing	Vitest (backend), pytest (AI backend)
UI Testing	React Testing Library (component level), Playwright (journey level)
Performance Profiling	Chrome DevTools / Lighthouse (frontend); NestJS request logging (backend)
Load Testing	autocannon — one manual baseline run on 2026-09-27; not automated (see Section 3.1.5)
Security & Access Control	Ownership-check unit tests, the e2e guard-chain tests, manual review of the AI backend's network exposure
Failover & Recovery	e2e_check.py --determinism, check_llm_chain_reliability.py; the 502-mapping unit tests
Configuration Testing	Three GitHub Actions workflows, one per repo

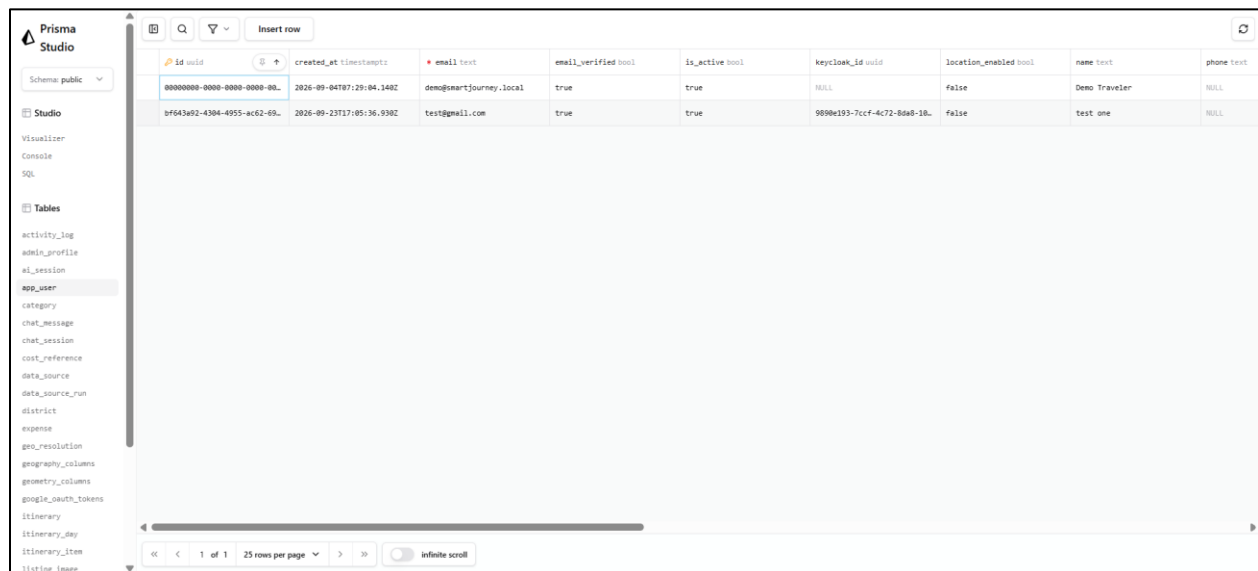
3.1 Testing Techniques and Types

3.1.1 Data and Database Integrity Testing

The database and its access layer are tested as an independent subsystem. Prisma's schema must match the hand-written SQL migrations that actually own the database, and every query's ownership/scoping filter must be correct.

Technique Objective:	Verify that Prisma's schema matches the hand-written SQL migrations that own the database, and that ownership/scoping filters in every query are correct.
Technique:	• Every backend service's unit spec constructs a hand-rolled Prisma fake and asserts on the exact where/data object passed to it. • budget.service.spec.ts asserts that getOwnedItinerary always filters by both id and user_id.

	Schema drift is caught by running prisma db pull and diffing against the checked-in schema.
Oracles:	The Prisma fake's recorded call arguments are the oracle for unit tests; psql / Prisma Studio against the live PostGIS container is the oracle for schema-vs-migration drift.
Required Tools:	- Vitest - Prisma Client types - psql / Prisma Studio (this project's equivalent of PHPMyAdmin / MySQL Workbench)
Success Criteria:	- No query in a spec'd module reaches Postgres without an explicit user_id (or admin-only) filter. - prisma db pull produces no unexpected diff.
Special Considerations:	Migrations are raw SQL, not Prisma Migrate - RUNNING.md warns that the migration runner's failures can be silent, so its exit code must always be checked.



The screenshot shows the Prisma Studio interface. On the left, a sidebar lists the database schema entities: activity_log, admin_profile, ai_session, app_user, category, chat_message, chat_session, cost_reference, data_source, data_source_run, district, expense, geo_resolution, geography_columns, geometry_columns, google_oauth_tokens, itinerary, itinerary_day, itinerary_item, and listing_image. The main area displays the 'app_user' table with two rows of seeded data. The table has columns: id (uuid), created_at (timestamp), email (text), email_verified (bool), is_active (bool), keycloak_id (uuid), location_enabled (bool), name (text), and phone (text). The first row has id '00000000-0000-0000-0000-00...', email 'demo@smartjourney.local', and name 'Demo Traveler'. The second row has id 'bf643a92-4304-4955-ac62-69...', email 'test@gmail.com', and name 'test one'.

id (uuid)	created_at (timestamp)	email (text)	email_verified (bool)	is_active (bool)	keycloak_id (uuid)	location_enabled (bool)	name (text)	phone (text)
00000000-0000-0000-0000-00...	2026-09-04T07:29:04.140Z	demo@smartjourney.local	true	true	NULL	false	Demo Traveler	NULL
bf643a92-4304-4955-ac62-69...	2026-09-23T17:05:36.930Z	test@gmail.com	true	true	9890e193-7ccf-4c72-8d08-18...	false	test one	NULL

Figure 1: Database schema: Prisma Studio showing the `app_user` table with seeded traveler accounts, alongside the core entity list (`app_user`, `itinerary`, `expense`, `travel_listing`, `local_event`) in the sidebar.

3.1.2 Function Testing

Function testing verifies each service method's business logic in isolation: status computation, ownership enforcement, idempotency, and error mapping.

Technique Objective:	Verify each service method's business logic in isolation - status computation, ownership enforcement, idempotency, and error mapping.
Technique:	• Vitest for backend: 69 unit tests across 14 files (verified passing 2026-09-26, re-run 2026-09-27). • pytest for the AI backend: 544 tests across 42 files (pre-existing, unmodified this round), run as <code>python -m pytest -m "not external" -q</code>. • AI-related unit tests shall use mocked or simulated LLM responses rather than live LLM calls. This ensures that tests remain deterministic, repeatable, independent of external API availability, and do not consume production or free-tier API quotas.
Oracles:	Hand-computed expected values in each assertion - e.g. <code>computeStatus</code> 's three budget thresholds, or <code>tripToItineraryDays</code> 's UTC time formatting.
Required Tools:	• Vitest 4 • pytest 9.1.1 • <code>@vitest/coverage-v8</code>
Success Criteria:	100% of unit tests pass with zero live services or API keys reachable.
Special Considerations:	The AI backend's LLM calls are always faked in unit tests; no unit test may make a real network call - verified by running the full suite with no Docker containers running. The first CI run showed that some of these tests silently relied on API keys from the local <code>.env</code> file (a key must be present before the mocked call is reached); CI now supplies placeholder keys (Section 3.1.8). Plain pytest cannot import the app package, so the suite is always run as <code>python -m pytest</code> .

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\Smart_Tourism\backend> npm test

> backend@0.0.1 test
> vitest run

The plugin "vite-tsconfig-paths" is detected. Vite now supports tsconfig paths resolution natively via the resolve.tsconfigPaths option. You can remove the
plugin and set resolve.tsconfigPaths: true in your Vite config instead.

v4.1.11 D:/Smart_Tourism/backend
✓ src/admin/admin-analytics.service.spec.ts (2 tests) 8ms
[Nest] 6204 - 09/27/2026, 4:32:35 PM ERROR [AdminContentService] Failed to write activity_log for listing.verify: Error: db down
✓ src/explore/explore.service.spec.ts (6 tests) 13ms
✓ src/trips/trips.service.spec.ts (6 tests) 14ms
✓ src/admin/admin-content.service.spec.ts (6 tests) 15ms
✓ src/budget/budget.service.spec.ts (11 tests) 17ms
[Nest] 18144 - 09/27/2026, 4:32:35 PM LOG [UserService] Provisioning app_user for Keycloak user kc-sub-1 (ann@example.com)
[Nest] 18144 - 09/27/2026, 4:32:35 PM LOG [UserService] Linking existing app_user legacy-1 to Keycloak user kc-sub-1
[Nest] 18144 - 09/27/2026, 4:32:35 PM LOG [UserService] Provisioning app_user for Keycloak user kc-sub-1 (ann@example.com)
✓ src/admin/admin-users.service.spec.ts (7 tests) 13ms
[Nest] 25980 - 09/27/2026, 4:32:35 PM ERROR [AiBackendService] AI backend /trip-plan call failed: Request failed with status code 500
AxiosError: Request failed with status code 500
    at D:/Smart_Tourism/backend/src/chat/ai-backend.service.spec.ts:37:28
    at file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:40
    at runWithSuite (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2258:8)
    at Object.collect (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:10)
    at Object.collect (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1893:54)
    at processTicksAndRejections (node:internal/process/task_queues:104:5)
    at file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2462:26
    at collectTests (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2433:3)
    at startTests (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:3280:17)
    at file:///D:/Smart_Tourism/backend/node_modules/vitest/dist/chunks/base.B60pl8PE.js:90:5
[Nest] 18144 - 09/27/2026, 4:32:35 PM LOG [UserService] Provisioning app_user for Keycloak user kc-sub-1 (ann@example.com)
[Nest] 18144 - 09/27/2026, 4:32:35 PM LOG [UserService] Provisioning app_user for Keycloak user kc-sub-1 (ann@example.com)
[Nest] 25980 - 09/27/2026, 4:32:35 PM ERROR [AiBackendService] AI backend /trip-plan call failed: timeout of 120000ms exceeded
AxiosError: timeout of 120000ms exceeded
    at D:/Smart_Tourism/backend/src/chat/ai-backend.service.spec.ts:38:19
    at file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:40

AxiosError: timeout of 120000ms exceeded
    at D:/Smart_Tourism/backend/src/chat/ai-backend.service.spec.ts:38:19
    at file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:40
    at runWithSuite (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2258:8)
    at Object.collect (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:10)
    at Object.collect (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1893:54)
    at processTicksAndRejections (node:internal/process/task_queues:104:5)
    at file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2462:26
    at collectTests (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2433:3)
    at startTests (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:3280:17)
    at file:///D:/Smart_Tourism/backend/node_modules/vitest/dist/chunks/base.B60pl8PE.js:90:5
[Nest] 25980 - 09/27/2026, 4:32:35 PM ERROR [AiBackendService] AI backend /trip-plan call failed: connect ECONNREFUSED
AxiosError: connect ECONNREFUSED
    at D:/Smart_Tourism/backend/src/chat/ai-backend.service.spec.ts:39:25
    at file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:40
    at runWithSuite (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2258:8)
    at Object.collect (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:10)
    at Object.collect (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1893:54)
    at processTicksAndRejections (node:internal/process/task_queues:104:5)
    at file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2462:26
    at collectTests (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2433:3)
    at startTests (file:///D:/Smart_Tourism/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:3280:17)
    at file:///D:/Smart_Tourism/backend/node_modules/vitest/dist/chunks/base.B60pl8PE.js:90:5
✓ src/users/users.service.spec.ts (6 tests) 36ms
✓ src/chat/ai-backend.service.spec.ts (5 tests) 17ms
✓ src/auth/jwt.strategy.spec.ts (2 tests) 7ms
✓ src/chat/chat.service.spec.ts (9 tests) 11ms
✓ src/auth/keycloak-token.spec.ts (2 tests) 3ms
✓ src/app.controller.spec.ts (1 test) 130ms
✓ src/auth/jwt-auth.guard.spec.ts (2 tests) 3ms
✓ src/auth/roles.guard.spec.ts (4 tests) 4ms

Test Files 14 passed (14)
Tests 69 passed (69)
Start at 16:32:34
Duration 1.20s (transform 995ms, setup 0ms, import 6.50s, tests 294ms, environment 2ms)

```

Figure 2: Unit test run (backend): Vitest run of all 14 backend spec files, 69 tests passing, including error-path tests for a failed activity-log write and unreachable AI backend.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Shaluka> cd D:\Smart_Tourism\ai-backend
PS D:\Smart_Tourism\ai-backend> .venv\Scripts\Activate.ps1
(.venv) PS D:\Smart_Tourism\ai-backend> python -m pytest -m "not external" -q
..... [ 20%]
..... [ 41%]
..... [ 61%]
..... [ 82%]
..... [100%]
===== warnings summary =====
.venv\Lib\site-packages\google\genai\types.py:42
D:\Smart_Tourism\ai-backend\.venv\Lib\site-packages\google\genai\types.py:42: DeprecationWarning: '_UnionGenericAlias' is deprecated and slated for removal in Python 3.17
  VersionedUnionType = Union[builtin_types.UnionType, _UnionGenericAlias]

.venv\Lib\site-packages\fastapi\testclient.py:1
D:\Smart_Tourism\ai-backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
544 passed, 2 warnings in 0.82s

```

Figure 3: Unit test run (AI backend): pytest run of the AI backend's non-external test suite, 544 tests passing with 2 deprecation warnings.

3.1.3 User Interface Testing

UI testing verifies route protection, token-refresh behaviour, and component rendering without a live stack, and real user journeys against a live stack when one is available.

Technique Objective:	Verify route protection (middleware), token-refresh behaviour, and component rendering; verify real user journeys against a live stack.
Technique:	- React Testing Library for component/logic-level tests: 5 files, 18 tests covering the middleware's role-gating logic, the Keycloak JWT-refresh callback, the trip mapper, the trip store, and one component. - Playwright for journey-level tests against a live Next.js dev server plus the full Docker/Keycloak/backend/AI-backend stack.
Oracles:	Rendered-DOM assertions (jest-dom matchers) for component tests; URL and visible-element assertions for Playwright.
Required Tools:	- Vitest - React Testing Library - jsdom - Playwright
Success Criteria:	- All RTL tests pass with no backend running at all. - Playwright's unauthenticated-redirect scenario passes against a bare "next dev" server.
Special Considerations:	Playwright's three authenticated scenarios (home renders the chat panel, a saved trip appears, a non-admin is blocked from /admin) need a seeded Keycloak session that this repo has no scripted way to obtain headlessly yet. They are written and present in

e2e/journey.spec.ts, marked test.skip with the reason stated inline. Only the one scenario that needs no auth was verified actually passing.

```
(.venv) PS D:\Smart_Tourism> cd .\frontend-web\  
(.venv) PS D:\Smart_Tourism\frontend-web> npm test  
  
> smartjourney-frontend@0.1.0 test  
> vitest run  
  
The plugin "vite-tsconfig-paths" is detected. Vite now supports tsconfig paths resolution natively via the resolve.tsconfigPaths option. You can remove the plugin and set resolve.tsconfigPaths: true in your Vite config instead.  
  
v4.1.11 D:/Smart_Tourism/frontend-web  
✓ src/lib/__tests__/trip-mappers.spec.ts (4 tests) 4ms  
✓ src/lib/__tests__/auth.spec.ts (4 tests) 5ms  
✓ src/lib/__tests__/authz.spec.ts (5 tests) 3ms  
✓ src/lib/__tests__/trip-store.spec.ts (3 tests) 4ms  
✓ src/components/budget/__tests__/SpendByCategoryPanel.spec.tsx (2 tests) 54ms  
  
Test Files 5 passed (5)  
Tests 18 passed (18)  
Start at 16:42:51  
Duration 2.01s (transform 225ms, setup 1.76s, import 336ms, tests 71ms, environment 6.68s)
```

Figure 4: Frontend unit test run

```
(.venv) PS D:\Smart_Tourism\frontend-web> npm test  
  
> smartjourney-frontend@0.1.0 test  
> vitest run  
  
The plugin "vite-tsconfig-paths" is detected. Vite now supports tsconfig paths resolution natively via the resolve.tsconfigPaths option. You can remove the plugin and set resolve.tsconfigPaths: true in your Vite config instead.  
  
v4.1.11 D:/Smart_Tourism/frontend-web  
✓ src/lib/__tests__/trip-mappers.spec.ts (4 tests) 4ms  
✓ src/lib/__tests__/auth.spec.ts (4 tests) 5ms  
✓ src/lib/__tests__/authz.spec.ts (5 tests) 3ms  
✓ src/lib/__tests__/trip-store.spec.ts (3 tests) 4ms  
✓ src/components/budget/__tests__/SpendByCategoryPanel.spec.tsx (2 tests) 54ms  
  
Test Files 5 passed (5)  
Tests 18 passed (18)  
Start at 16:42:51  
Duration 2.01s (transform 225ms, setup 1.76s, import 336ms, tests 71ms, environment 6.68s)  
  
(.venv) PS D:\Smart_Tourism\frontend-web> npm run test:e2e  
  
> smartjourney-frontend@0.1.0 test:e2e  
> playwright test  
  
Running 4 tests using 4 workers  
3 skipped  
1 passed (14.5s)  
  
To open last HTML report run:  
npx playwright show-report  
  
(.venv) PS D:\Smart_Tourism\frontend-web> npx playwright show-report  
Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.  
|
```

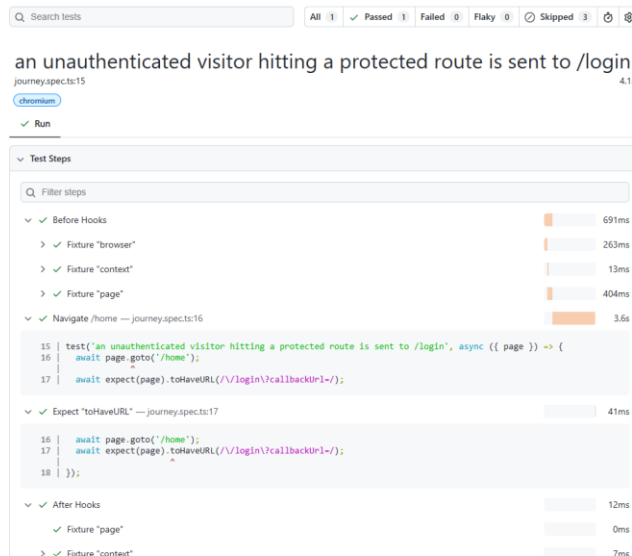


Figure 5: Playwright HTML report

3.1.4 Performance Profiling

Technique Objective:	Verify response times and resource usage for both the API and the browser under normal load.
Technique:	• Chrome DevTools' Performance and Network panels for page-load cost. • NestJS request logs for API latency. • The AI backend's documented 5-20 second typical /trip-plan latency as the baseline the 120-second NestJS timeout is set against.
Oracles:	No formally agreed SLA exists yet. The only enforced number is the 120-second NestJS-to-AI-backend timeout, tested by the timeout-mapping case in ai-backend.service.spec.ts.
Required Tools:	• Chrome DevTools • Lighthouse
Success Criteria:	/trip-plan responses complete within the 120-second budget, or degrade to the deterministic fallback path (Section 3.1.7), never hanging indefinitely.
Special Considerations:	This is the least mature area of the plan - there is no dashboard yet. A first page-load baseline was recorded on 2026-09-27 (Figure 6: 31 requests, DOMContentLoaded 211 ms, load 600 ms on /home), measured against the next dev server rather than a

production build, so it is a reference point, not an SLA. Flagged as a gap in Section 5, not glossed over.

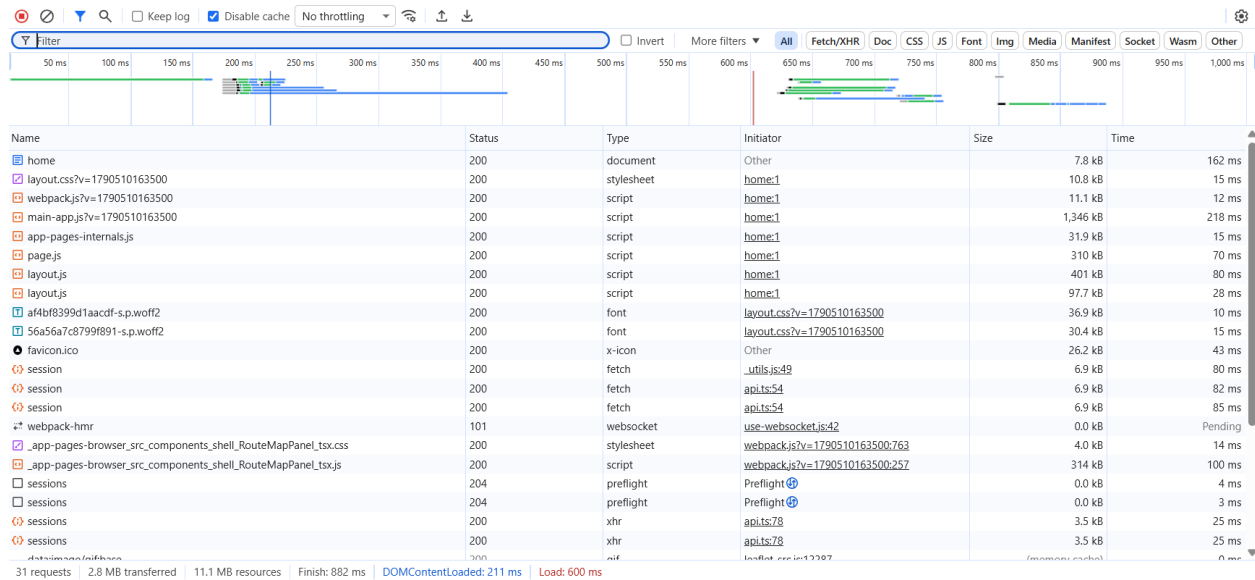


Figure 6: Page load performance: Chrome DevTools waterfall for /home, showing 31 requests (2.8 MB transferred), DOMContentLoaded at 211 ms and full load at 600 ms

3.1.5 Load Testing

Technique Objective:	Determine behaviour under concurrent users, particularly against the free-tier external APIs the AI backend depends on.
Technique:	Not yet automated. One manual baseline run on 2026-09-27: <code>npx autocannon -c 20 -d 30 http://localhost:3001/listings</code> (20 connections for 30 seconds against the public, read-heavy listings route). The AI backend's own <code>check_llm_chain_reliability.py</code> already exercises the Groq/Gemini failover chain under repeated calls.
Oracles:	No formal pass/fail threshold is defined yet.
Required Tools:	autocannon, run via npx (not a dependency of any repo)
Success Criteria:	Baseline recorded 2026-09-27 (Figure 7): avg 394 req/sec, p50 latency 50 ms, p99 68 ms, max 145 ms, 12k requests in 30 s. Pass/fail thresholds are still to be set against this baseline.
Special Considerations:	For AI-related load testing, external service limitations shall be considered separately from the application's own processing capacity. External API rate limits and LLM token quotas shall not be unnecessarily consumed during automated load tests; therefore, mocked responses should be used where appropriate to evaluate application behaviour under controlled concurrent workloads.

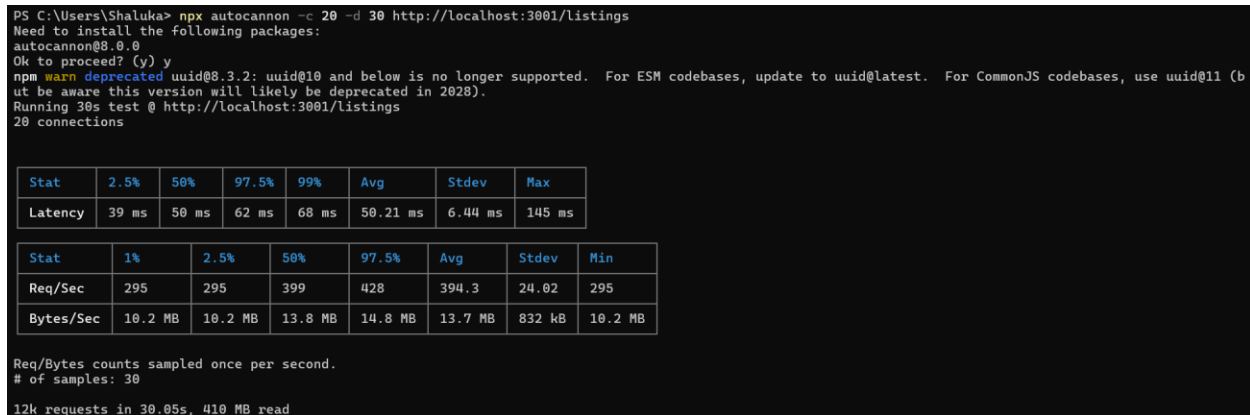


Figure 7: Load test run: autocannon against /listings with 20 concurrent connections over 30 s - avg 394 req/sec, p50 latency 50 ms, p99 68 ms, 12k requests / 410 MB read total

3.1.6 Security and Access Control Testing

Unlike a system that can say "SQL injection is neglected since the ORM manages access", this project cannot make the equivalent claim about the AI backend's exposure: it has no authentication of its own, so NestJS's Keycloak-backed guards are the only trust boundary in the whole system.

Technique Objective:	Verify that (a) a traveler can only ever act on their own data, (b) role-gated routes correctly reject the wrong role, and (c) the AI backend is never reachable except through NestJS.
Technique:	- Every backend service spec includes an explicit "foreign user gets NotFoundException" case (not ForbiddenException - deliberately indistinguishable from "doesn't exist"). - roles.guard.spec.ts covers the role-based-access decision table. - The end-to-end suite exercises the real JwtAuthGuard -> RolesGuard -> CurrentUser chain using a signed JWT against a substitute Keycloak strategy.
Oracles:	HTTP status codes (401 unauthenticated, 403 wrong role, 404 wrong owner) and the exact Prisma "where" clause used.
Required Tools:	- Vitest - supertest - jsonwebtoken (for signing test tokens) - manual check that the AI backend's port is not exposed outside the Docker network

Success Criteria:	100% of the ownership and role-based-access unit/e2e tests pass; the AI backend is confirmed unreachable from outside the Docker network in the deployed topology.
Special Considerations:	The AI backend accepts requests from anyone who can reach it, with no token check. The mitigation is entirely at the infrastructure layer (Docker network isolation / firewall), not the application layer, and is not covered by any automated test in this plan - flagged as a residual risk in Section 5.

```

PS C:\Users\Shaluka> cd D:\Smart_Tourism\backend
PS D:\Smart_Tourism\backend> npm run test:e2e

> backend@0.0.1 test:e2e
> vitest run --config ./vitest.config.e2e.ts

The plugin "vite-tsconfig-paths" is detected. Vite now supports tsconfig paths resolution natively via the resolve.tsconfigPaths option. You can remove the
plugin and set resolve.tsconfigPaths: true in your Vite config instead.

v4.1.11 D:/Smart_Tourism/backend

✓ test/app.e2e-spec.ts (6 tests) 211ms
✓ App (e2e) (6)
  ✓ GET / is public and needs no identity 132ms
  ✓ a protected route with no bearer token is rejected with 401 12ms
  ✓ a traveler hitting /admin/stats gets 403, not a peek at the data 14ms
  ✓ POST /trips rejects an unknown field instead of silently dropping it 28ms
  ✓ GET /trips returns the caller's own trips 11ms
  ✓ GET /budget/summary returns [] for a traveler with no trips yet 11ms

Test Files 1 passed (1)
Tests 6 passed (6)
Start at 17:34:20
Duration 1.56s (transform 328ms, setup 0ms, import 1.22s, tests 211ms, environment 0ms)

```

Figure 8: Backend e2e security tests: All 6 end-to-end guard-chain tests passing, including a 403 for a traveler hitting /admin/stats and a 401 for a request with no bearer token.

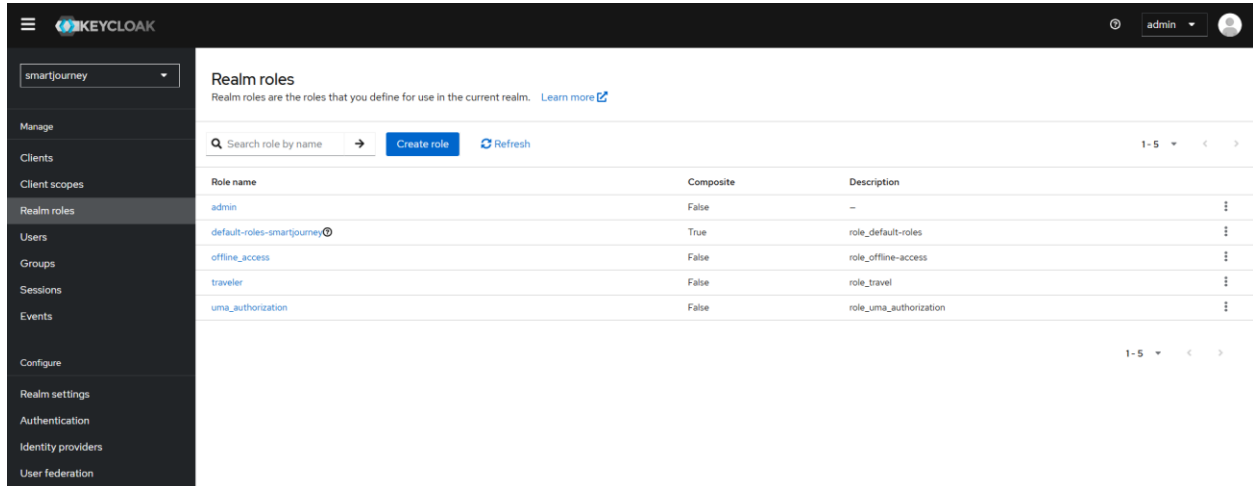


Figure 9: Keycloak realm roles: Realm roles page for the smartjourney realm, showing the admin and traveler roles used by RolesGuard

3.1.7 Failover and Recovery Testing

This section verifies that the AI system handles LLM and external data-source failures gracefully. When an AI service becomes unavailable, times out, or returns an invalid response, the system shall detect the failure and activate the defined fallback mechanism without exposing an unhandled exception to the user.

Technique Objective:	Verify the system degrades gracefully when the LLM or an external data source is unavailable.
Technique:	- ai-backend.service.spec.ts asserts a non-2xx response, a timeout, and a network error from the AI backend all map to a 502 in NestJS, never an unhandled exception. - At the AI backend layer, the pre-existing fallback tests and the golden-scenario harness (scripts/e2e_check.py) cover the LLM-unavailable path, verifying a fallback plan and HTTP 200 rather than an error.
Oracles:	For LLM-dependent functionality, test oracles shall evaluate structural and functional properties rather than exact generated text. The generated itinerary shall be checked for schema validity, budget compliance, required output fields, and the correct identification of the plan source. Different outputs for the same prompt shall be considered acceptable when they satisfy the defined validation criteria.
Required Tools:	- pytest - scripts/e2e_check.py -determinism - scripts/check_llm_chain_reliability.py
Success Criteria:	Golden scenario "Gemini unavailable" passes with a fallback plan and HTTP 200 (confirmed passing again in the 2026-09-27 live run, Figure 10).
AI-Specific Quality Criteria	AI-generated itineraries shall satisfy the required output schema, remain within the user's specified budget where applicable, respect applicable system constraints and policies, and provide a valid fallback response when the primary LLM is unavailable. Follow-up requests shall preserve relevant context from the previous interaction without causing an unhandled exception.
Special Considerations:	The 2026-09-27 live run scored 10/12 (83.3%), below the $\geq 11/12$ target, and supersedes the 11/12 recorded on 2026-09-03. The first attempt that day scored 8/12 because the harness itself was broken (stale mock-patch targets, fixed – Section 4.1). After the fix, scenario 5 (follow-up "make day 2 cheaper") failed its "day 2 cost strictly lower" check, and scenario 6 (follow-up "I'm starting from Polonnaruwa") raised an unhandled exception - an open orchestrator defect (Section 4.1), triggered by Gemini 504 timeouts but not caused by them. Recorded as measured rather than rounded up.

```
PS C:\Users\Shaluka> cd D:\Smart_Tourism\ai-backend
PS D:\Smart_Tourism\ai-backend> .venv\Scripts\Activate.ps1
(.venv) PS D:\Smart_Tourism\ai-backend> python scripts\e2e_check.py
    running #1 - Handy 3-day, budget LKR 60000, culture/history ...
D:\Smart_Tourism\ai-backend\.venv\Lib\site-packages\langchain_google_genai\chat_models.py:3120: UserWarning: Model 'gemini-3.5-flash-lite' uses fixed sampling defaults; the sampling parameter(s) temperature, top_k, top_p will be ignored.
  request = self._build_request_config()
D:\Smart_Tourism\ai-backend\.venv\Lib\site-packages\langchain_google_genai\chat_models.py:3120: UserWarning: Model 'gemini-3.5-flash-lite' uses fixed sampling defaults; the sampling parameter(s) temperature, top_k, top_p will be ignored.
  request = self._build_request_config()
D:\Smart_Tourism\ai-backend\.venv\Lib\site-packages\langchain_google_genai\chat_models.py:3120: UserWarning: Model 'gemini-3.5-flash-lite' uses fixed sampling defaults; the sampling parameter(s) temperature, top_k, top_p will be ignored.
  request = self._build_request_config()
D:\Smart_Tourism\ai-backend\.venv\Lib\site-packages\langchain_google_genai\chat_models.py:3120: UserWarning: Model 'gemini-3.5-flash-lite' uses fixed sampling defaults; the sampling parameter(s) temperature, top_k, top_p will be ignored.
  request = self._build_request_config()
D:\Smart_Tourism\ai-backend\.venv\Lib\site-packages\langchain_google_genai\chat_models.py:3120: UserWarning: Model 'gemini-3.5-flash-lite' uses fixed sampling defaults; the sampling parameter(s) temperature, top_k, top_p will be ignored.
  request = self._build_request_config()
D:\Smart_Tourism\ai-backend\.venv\Lib\site-packages\langchain_google_genai\chat_models.py:3120: UserWarning: Model 'gemini-3.5-flash-lite' uses fixed sampling defaults; the sampling parameter(s) temperature, top_k, top_p will be ignored.
  request = self._build_request_config()
D:\Smart_Tourism\ai-backend\.venv\Lib\site-packages\langchain_google_genai\chat_models.py:3120: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature, top_k, top_p will be ignored.
  request = self._build_request_config()
D:\Smart_Tourism\ai-backend\.venv\Lib\site-packages\langchain_google_genai\chat_models.py:3120: UserWarning: Model 'gemini-3.5-flash-lite' uses fixed sampling defaults; the sampling parameter(s) temperature, top_k, top_p will be ignored.
  request = self._build_request_config()
D:\Smart_Tourism\ai-backend\.venv\Lib\site-packages\langchain_google_genai\chat_models.py:3120: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature, top_k, top_p will be ignored.
  request = self._build_request_config()
PlannerAgent failed: run_react's finalize call for PlannerOutput failed to parse (loop stopped by=timeout): Failed to parse PlannerOutput from completion {"itinerary": [{"day": "1", "date": "2026-09-27", "theme": "Cultural & Historical Exploration", "items": [{"time": "09:18", "end_time": "09:48", "type": "hotel"}, {"listing_id": "01309e-5272-4f9d-b5b8-7a7991064bb8", "name": "Forest Villa", "lat": "7.2843874", "lon": "80.6446291", "est_cost": 39585.69, "currency": "LKR", "notes": ""}, {"time": "09:58", "end_time": "11:13", "type": "attraction", "listing_id": "1ee93e97-f74f-4d80e-8a32-f292d5b5c5d3", "name": "Buddha", "lat": "7.3043239", "lon": "80.6431704", "est_cost": 0.0, "currency": "LKR", "notes": ""}, {"time": "11:27", "end_time": "12:57", "type": "attraction", "listing_id": "25c15260-155c-4d3c-8dc9-bb3d64509891", "name": "Thwaites monument", "lat": "7.2721848", "lon": "80.5966635", "est_cost": 0.0, "currency": "LKR", "notes": ""}, {"time": "13:10", "end_time": "14:10", "type": "restaurant", "listing_id": "129891b5-c887-459c-9d8a-5bfdf710084b", "name": "Vijayan's Take Away", "lat": "7.22
```

Figure 10: Golden scenario run: 10/12 passing (target $\geq 11/12$) after fixing stale mock-patch targets in the harness; scenario 5 fails the day-2 cost check and scenario 6 hits an open orchestrator defect (Section 4.1).

Technique Objective:	Verify each service’s test suite runs correctly in a clean environment with no locally-installed extras and no secrets.
Technique:	Three GitHub Actions workflows (one per repo), each a from-scratch checkout, dependency install, and test run, with zero Docker services and no real secrets (the ai-backend workflow supplies placeholder API keys, since key-gated code runs before the mocked calls).

Oracles:	The CI job's exit code.
Required Tools:	• GitHub Actions • Node 26 (backend), Node 22 (frontend-web) • Python 3.12
Success Criteria:	All three workflows pass on a clean checkout - green on 2026-09-27 (Figures 11–13). Reproducing the commands locally beforehand was not enough: all three failed on their first push, for configuration reasons only. Fixes: backend moved to Node 26 because Node 22's npm 10 rejected the npm-11 lockfile; ai-backend runs <code>python -m pytest</code> , adds <code>respx</code> , <code>itsdangerous</code> and <code>pytest-asyncio</code> to <code>requirements.txt</code> , and supplies placeholder API keys; frontend-web's push trigger now covers the new-main branch as well as main.
Special Considerations:	Coverage is reported, not gated - no minimum-coverage threshold is enforced yet, since setting one on a suite this young would fail the build for the wrong reason. Playwright is deliberately excluded from CI (see Section 3.1.3).

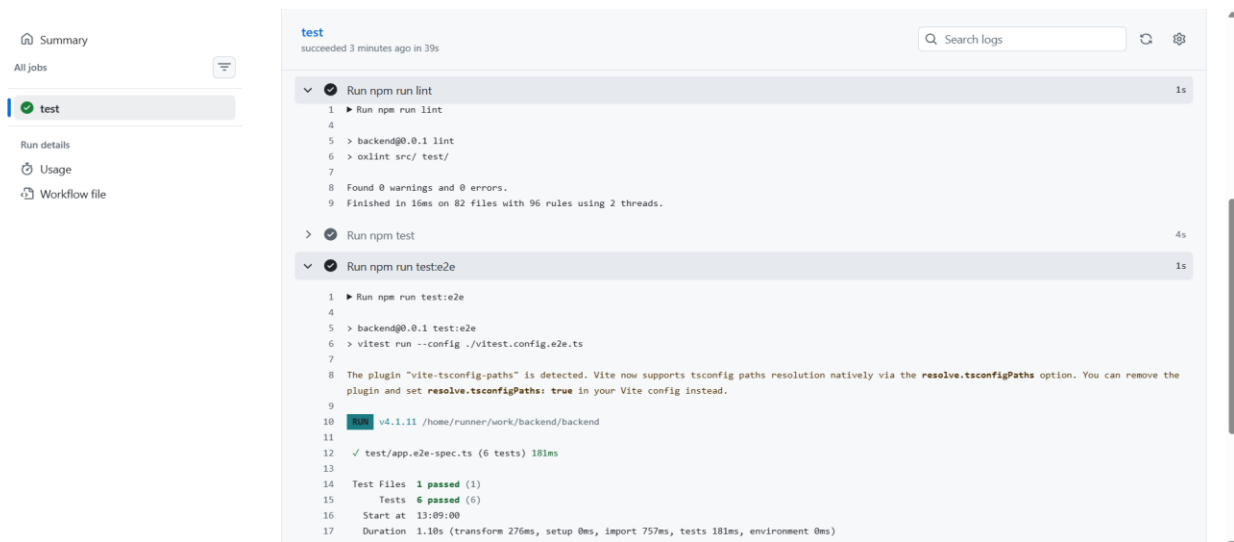


Figure 11: CI runs (backend): GitHub Actions run on push, lint passing with 0 errors/warnings and the e2e suite passing 6/6 tests.

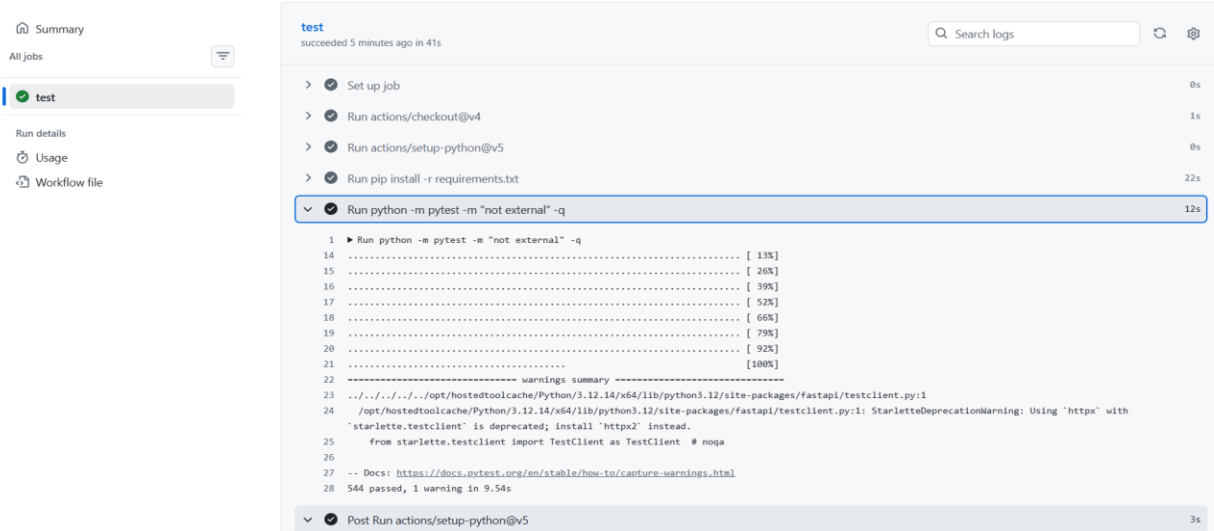


Figure 12: CI runs (ai-backend): GitHub Actions run on push, pytest passing 544/544 tests with 1 warning, using placeholder API keys since no test makes a live network call.

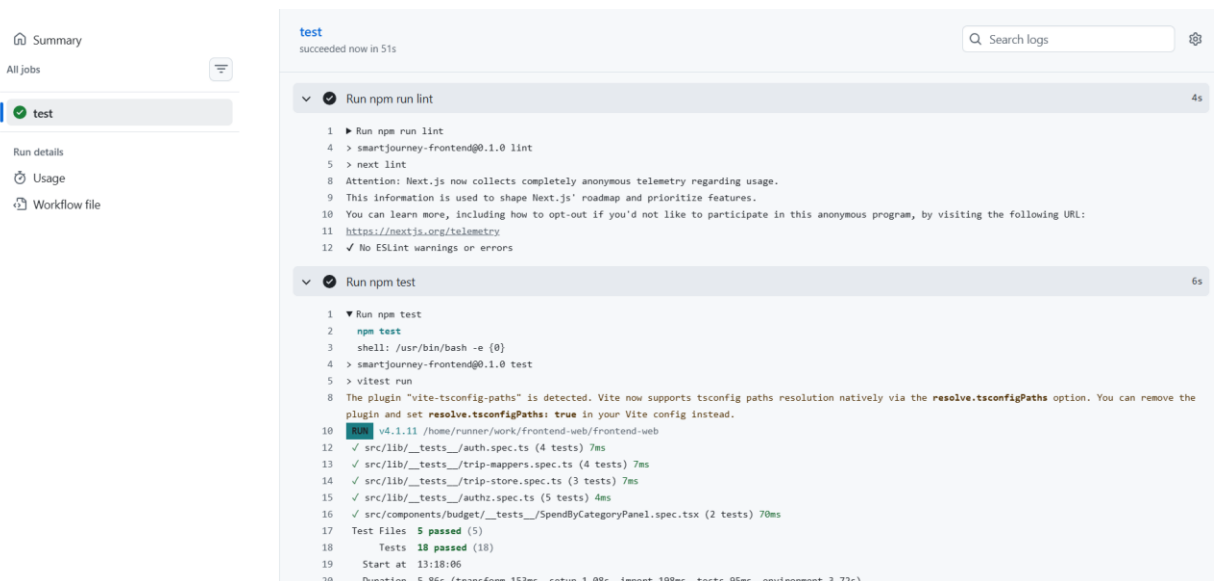


Figure 13: CI runs (frontend-web): GitHub Actions run on push, ESLint passing with no warnings or errors and Vitest passing all 5 test files (18/18 tests)

4. Deliverables

The artifacts produced by this test effort, and how often each is refreshed:

- Test logs - the console output of every Vitest/pytest run.
- Coverage reports - the v8/pytest-cov summary produced by the "test:cov" / "--cov" commands.
- User Interface test details - the Playwright HTML report.
- Code inspection output - oxlint (backend) and next lint (frontend) results.
- Database schema/performance snapshots - from Prisma Studio / psql.
- CI run history - the GitHub Actions log for each repo.

4.1 Test Evaluation Summary (as actually run, 2026-09-27)

Repo	Command	Result
backend	npm run lint	0 errors (oxlint)
backend	npm test	69 passed, 0 failed - 14 spec files
backend	npm run test:cov	70.37% statements / 53.35% branches / 62.58% functions / 72.30% lines
backend	npm run test:e2e	6 passed, 0 failed
backend	npx autocannon -c 20 -d 30 http://localhost:3001/listings	avg 394 req/sec, p50 50 ms, p99 68 ms, 12k requests in 30 s (20 connections, local)
frontend-web	npm run lint	0 errors/warnings (next lint)
frontend-web	npm test	18 passed, 0 failed - 5 spec files
frontend-web	npm run test:cov	75.34% statements / 65.90% branches / 83.33% functions / 76.92% lines
frontend-web	npx playwright test	1 passed (unauthenticated redirect), 3 skipped (need a seeded Keycloak session) - from the 2026-09-26 run, not re-run 2026-09-27
ai-backend	python -m pytest -m "not external" -q	544 passed, 0 failed - 42 files (pre-existing suite, unmodified)
ai-backend	scripts/e2e_check.py	10/12 (83.3%) golden scenarios, live run 2026-09-27 - below the $\geq 11/12$ target; see section 3.1.7

% Coverage report from v8					
File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	70.37	53.35	62.58	72.3	
src	100	50	100	100	
app.controller.ts	100	50	100	100	5
app.service.ts	100	100	100	100	
src/admin	50.22	30.87	46.03	52.5	
admin-analytics.service.ts	65.15	31.57	50	75	105-109,123-124,136-137,155-160
admin-content.service.ts	40.5	22.05	42.85	39.47	22-24,48-59,121-152,176-287
admin-users.service.ts	85.71	71.42	85.71	87.17	107-108,139-140,154
keycloak-admin.service.ts	5.26	0	0	6.06	17-91
src/auth	90.24	76.92	83.33	91.42	
current-user.decorator.ts	33.33	0	0	33.33	11-12
index.ts	0	0	0	0	
jwt-auth.guard.ts	100	75	100	100	10
jwt.strategy.ts	90	75	100	90	23
keycloak-token.ts	100	100	100	100	
public.decorator.ts	100	100	100	100	
roles.decorator.ts	66.66	100	0	100	
roles.guard.ts	100	90	100	100	11
src/budget	93.75	78.33	100	93.02	
budget.service.ts	93.75	78.33	100	93.02	61,97-98
src/chat	93.18	73.91	86.66	93.02	
ai-backend.service.ts	100	50	100	100	14
chat.service.ts	90.9	76.19	83.33	90.62	15-21,70
src/explore	76.19	80	62.5	76.19	
explore.service.ts	76.19	80	62.5	76.19	18-31,90,122
src/generated/prisma	100	100	100	100	
client.ts	100	100	100	100	
enums.ts	0	0	0	0	
src/generated/prisma/internal	90.47	100	25	90.47	
class.ts	45.45	100	25	45.45	44-46,50-54
prismaNamespace.ts	100	100	100	100	
src/prisma	25	100	0	25	
prisma.service.ts	25	100	0	25	20-33
src/trips	81.81	71.11	100	86.2	
trips.service.ts	81.81	71.11	100	86.2	110,115,130-131
src/users	69.33	41.37	42.85	72.46	
users.service.ts	69.33	41.37	42.85	72.46	58,116-181

Figure 14: backend coverage report: 70.4% statements, 53.4% branch, 62.6% functions, 72.3% lines overall.

PS D:\Smart_Tourism\frontend-web> npm run test:cov					
> smartjourney-frontend@0.1.0 test:cov					
> vitest run --coverage					
The plugin "vite-tsconfig-paths" is detected. Vite now supports tsconfig paths resolution natively via the resolve.tsconfigPaths option. You can remove the plugin and set resolve.tsconfigPaths: true in your Vite config instead.					
v4.1.11 D:/Smart_Tourism/frontend-web					
Coverage enabled with v8					
✓ src/lib/__tests__/trip-mappers.spec.ts (4 tests) 5ms					
✓ src/lib/__tests__/auth.spec.ts (4 tests) 7ms					
✓ src/lib/__tests__/authz.spec.ts (5 tests) 6ms					
✓ src/lib/__tests__/trip-store.spec.ts (3 tests) 5ms					
✓ src/components/budget/__tests__/SpendByCategoryPanel.spec.tsx (2 tests) 62ms					
Test Files 5 passed (5)					
Tests 18 passed (18)					
Start at 18:52:10					
Duration 2.14s (transform 333ms, setup 2.01s, import 410ms, tests 85ms, environment 6.61s)					
% Coverage report from v8					
File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	75.34	65.9	83.33	76.92	
src	100	87.5	100	100	
middleware.ts	100	87.5	100	100	14
src/components/budget	100	100	100	100	
SpendByCategoryPanel.tsx	100	100	100	100	
src/lib	71.42	58.82	80	73.21	
auth.ts	61.7	50	50	65.11	54,60,100-124,163,183-192
trip-mappers.ts	100	100	100	100	
trip-store.ts	100	100	100	100	

Figure 15: frontend-web coverage report: 75.3% statements, 65.9% branch, 83.3% functions, 76.9% lines overall

Real defects found during this test effort (2026-09-26 and 2026-09-27):

- `explore.service.ts` `searchEvents` (fixed): the Prisma "where" clause was built with two separate object spreads both keyed `start_datetime` (one for the "from" bound, one for "to"); the second silently overwrote the first, so a request supplying both bounds dropped the "from" filter. Caught by `explore.service.spec.ts`'s date-window test, fixed by merging both bounds into one spread, and reverified passing.
- `scripts/e2e_check.py` (fixed): scenarios 7-9 patched `app.tools.registry.get_weather / get_disaster_info / get_free_days`, which an earlier refactor had moved out of the registry, so all three crashed before testing anything. They now patch the functions where they are used (`app.core.context_resolver`).
- Orchestrator follow-up crash (open): on a follow-up turn, when the planner fails (here after Gemini 504 timeouts), `_verify_node` in `app/core/orchestrator.py` still sees the previous turn's itinerary, skips its "no plan produced" guard, and calls `PlannerOutput.model_validate(None)`, which raises. Reproduced by golden scenario 6; the request should degrade to the fallback plan instead.
- CI configuration (fixed): the first-push failures in all three repos, described in section 3.1.8.

4.2 Reporting on Test Coverage

Unit tests (backend, frontend-web, ai-backend) run on every push via CI (Section 3.1.8) and locally on every relevant change. Coverage reports are generated on demand, not gated. Playwright's authenticated scenarios and the AI backend's golden-scenario harness are run manually against a live stack before any release, following `RUNNING.md`'s startup order.

Appendix A gives a generic manual test-log template (for UAT, exploratory testing, or the Playwright scenarios that need a live stack), in the same format the original sample report used.

4.3 Security and Access Control Test Cases

Section 3.1.6 sets out the approach to security and access control; this section lists the individual cases and what happened when they were run. The cases live in two end-to-end spec files in the backend repository and run under the Vitest e2e configuration. Each suite has its own npm script, and both also run as part of the full e2e command.

Command	Scope	Result (2026-09-27)
<code>npm run test:security</code>	<code>test/security.e2e-spec.ts</code>	8 passed, 0 failed
<code>npm run test:access</code>	<code>test/access-control.e2e-spec.ts</code>	9 passed, 0 failed
<code>npm run test:e2e</code>	All 3 e2e spec files: security (8), access control (9), app (6)	23 passed, 0 failed
<code>npm test</code>	All 14 unit spec files	69 passed, 0 failed

4.3.1 Security testing — authentication

This suite verifies the authentication boundary of the system. The AI backend has no authentication of its own (Section 3.1.6), so every request that reaches it has already passed through the NestJS guard chain - `JwtAuthGuard` followed by `RolesGuard` - and that chain is the only trust boundary in the system. The suite sends requests with no credentials, forged credentials, expired credentials and malformed credentials, and checks that each is refused with 401 rather than let through or turned into a server error.

Three test cases provide key evidence for the authentication behaviour. SEC-02 shows that a token signed with the wrong key is rejected, so a signature cannot be faked. SEC-03 shows that writing the ADMIN role into a forged token gains nothing, so a caller cannot grant themselves a role. SEC-07 shows that a correctly signed token is accepted; without it, the rejections above could equally come from a guard that refuses everything.

ID	Test case	Result
SEC-01	A protected route with no credentials is rejected with 401	Pass
SEC-02	A forged token signed with the wrong key is rejected with 401	Pass
SEC-03	A forged ADMIN token cannot reach an admin route	Pass
SEC-04	An expired token is rejected with 401	Pass
SEC-05	A malformed token is rejected with 401, not a 500	Pass
SEC-06	A token sent without the Bearer scheme is rejected with 401	Pass
SEC-07	A validly signed token is accepted (the negatives above are not false positives)	Pass
SEC-08	A public route stays reachable without any identity	Pass

The run below was captured on 2026-09-27 by executing `npm run test:security` in the backend repository, with no services running.

```

(.venv) PS C:\Users\VICTUS\Desktop\SmartJourney-SmartTourismProject> cd backend
(.venv) PS C:\Users\VICTUS\Desktop\SmartJourney-SmartTourismProject\backend> npm run test:security

> backend@0.0.1 test:security
> vitest run --config ./vitest.config.e2e.ts test/security.e2e-spec.ts --reporter=verbose

 RUN v4.1.11 C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend

✓ test/security.e2e-spec.ts > Security testing - authentication (system-level) > SEC-01: a protected route with no credentials is rejected with 401 176ms
✓ test/security.e2e-spec.ts > Security testing - authentication (system-level) > SEC-02: a forged token signed with the wrong key is rejected with 401 16ms
✓ test/security.e2e-spec.ts > Security testing - authentication (system-level) > SEC-03: a forged ADMIN token cannot reach an admin route 8ms
✓ test/security.e2e-spec.ts > Security testing - authentication (system-level) > SEC-04: an expired token is rejected with 401 9ms
✓ test/security.e2e-spec.ts > Security testing - authentication (system-level) > SEC-05: a malformed token is rejected with 401, not a 500 8ms
✓ test/security.e2e-spec.ts > Security testing - authentication (system-level) > SEC-06: a token sent without the Bearer scheme is rejected with 401 7ms
✓ test/security.e2e-spec.ts > Security testing - authentication (system-level) > SEC-07: a validly signed token is accepted (the negatives above are not false positives) 15ms
✓ test/security.e2e-spec.ts > Security testing - authentication (system-level) > SEC-08: a public route stays reachable without any identity 7ms

Test Files 1 passed (1)
Tests 8 passed (8)
Start at 20:43:18
Duration 3.94s (transform 684ms, setup 0ms, import 3.10s, tests 519ms, environment 0ms)

```

Figure 16: Security test suite - all 8 authentication cases passing (SEC-01 to SEC-08), including a forged admin token rejected with 401 and a malformed token returning 401 rather than a server error.

4.3.2 Access control testing - authorization

Section 4.3.1 verifies authentication, while this suite verifies authorization and ownership controls. It covers role checks (a traveler against admin routes) and ownership checks (one traveler against another traveler's trips).

Two design choices are verified explicitly. A traveler who asks for someone else's trip gets 404 rather than 403, so the API does not confirm that the trip exists (AC-05); AC-06 then reads the same trip as its owner, which shows the 404 comes from scoping and not from a broken route. AC-09 confirms that the admin role does not bypass ownership on a traveler's own routes.

ID	Test case	Result
AC-01	A traveler is refused an admin route with 403	Pass

ID	Test case	Result
AC-02	A traveler is refused admin user management with 403	Pass
AC-03	An admin is granted the same admin route	Pass
AC-04	A traveler sees only their own trips in a list	Pass
AC-05	A traveler reading another traveler's trip gets 404, not the row	Pass
AC-06	The owner can read the very same trip (the 404 above is scoping, not breakage)	Pass
AC-07	A traveler cannot modify another traveler's trip	Pass
AC-08	A traveler cannot delete another traveler's trip	Pass
AC-09	An admin is not exempt from ownership on a traveler's own routes	Pass

The run below was captured on 2026-09-27 by executing `npm run test:access` in the backend repository, with no services running.

```

(.venv) PS C:\Users\VICTUS\Desktop\SmartJourney-SmartTourismProject\backend> npm run test:access
> backend@0.0.1 test:access
> vitest run --config ./vitest.config.e2e.ts test/access-control.e2e-spec.ts --reporter-verbose

RUN v4.1.11 C:\Users\VICTUS\Desktop\SmartJourney-SmartTourismProject\backend
✓ test/access-control.e2e-spec.ts > Access control testing – authorization (application-level) > AC-01: a traveler is refused an admin route with 403 34ms
✓ test/access-control.e2e-spec.ts > Access control testing – authorization (application-level) > AC-02: a traveler is refused admin user management with 403 6ms
✓ test/access-control.e2e-spec.ts > Access control testing – authorization (application-level) > AC-03: an admin is granted the same admin route 6ms
✓ test/access-control.e2e-spec.ts > Access control testing – authorization (application-level) > AC-04: a traveler sees only their own trips in a list 14ms
✓ test/access-control.e2e-spec.ts > Access control testing – authorization (application-level) > AC-05: a traveler reading another traveler's trip gets 404, not the row 5ms
✓ test/access-control.e2e-spec.ts > Access control testing – authorization (application-level) > AC-06: the owner can read the very same trip (the 404 above is scoping, not breakage) 5ms
✓ test/access-control.e2e-spec.ts > Access control testing – authorization (application-level) > AC-07: a traveler cannot modify another traveler's trip 23ms
✓ test/access-control.e2e-spec.ts > Access control testing – authorization (application-level) > AC-08: a traveler cannot delete another traveler's trip 5ms
✓ test/access-control.e2e-spec.ts > Access control testing – authorization (application-level) > AC-09: an admin is not exempt from ownership on a traveler's own routes 5ms

Test Files 1 passed (1)
Tests 9 passed (9)
Start at 20:44:15
Duration 3.00s (transform 750ms, setup 0ms, import 3.00s, tests 390ms, environment 0ms)

```

Figure 17: Access control test suite — all 9 authorization cases passing (AC-01 to AC-09), including a cross-owner read returning 404 rather than the row, and an admin held to ownership on a traveler's own routes.

Both suites also run inside the full e2e command, and the unit suite was re-run the same evening. Those two runs are shown below.

```

(.venv) PS C:\Users\VICTUS\Desktop\SmartJourney-SmartTourismProject\backend> npm run test:e2e
> backend@0.0.1 test:e2e
> vitest run --config ./vitest.config.e2e.ts

RUN v4.1.11 C:\Users\VICTUS\Desktop\SmartJourney-SmartTourismProject\backend
✓ test/security.e2e-spec.ts (8 tests) 394ms
✓ test/access-control.e2e-spec.ts (9 tests) 439ms
✓ test/app.e2e-spec.ts (6 tests) 511ms
  ✓ GET / is public and needs no identity 328ms

Test Files 3 passed (3)
Tests 23 passed (23)
Start at 20:45:32
Duration 4.47s (transform 2.23s, setup 0ms, import 10.65s, tests 1.34s, environment 1ms)

```

Figure 18: Full backend e2e run (`npm run test:e2e`): 3 spec files and 23 tests passing - security (8), access control (9) and app (6).

The npm test output is too long for one screen, so it was captured in three consecutive parts, shown in order. The red ERROR lines are logged by unit tests that deliberately exercise failure paths - an activity-log write against a database that is down, and an AI backend that returns 500, times out or refuses the connection - and are expected. The summary at the end shows every test passing.

```
(.venv) PS C:\Users\VICTUS\Desktop\SmartJourney-SmartTourismProject\backend> npm test

> backend@0.0.1 test
> vitest run

 RUN  v4.1.11 C:\Users\VICTUS\Desktop\SmartJourney-SmartTourismProject\backend

[Nest] 2056 - 09/27/2026, 8:46:13 PM ERROR [AdminContentService] Failed to write activity_log for listing.verify: Error: db down
✓ src/trips/trips.service.spec.ts (6 tests) 22ms
[Nest] 27640 - 09/27/2026, 8:46:13 PM LOG [UserService] Provisioning app_user for Keycloak user kc-sub-1 (ann@example.com)
✓ src/admin/admin-content.service.spec.ts (6 tests) 28ms
[Nest] 27640 - 09/27/2026, 8:46:13 PM LOG [UserService] Linking existing app_user legacy-1 to Keycloak user kc-sub-1
[Nest] 27640 - 09/27/2026, 8:46:13 PM LOG [UserService] Provisioning app_user for Keycloak user kc-sub-1 (ann@example.com)
✓ src/admin/admin-analytics.service.spec.ts (2 tests) 17ms
✓ src/budget/budget.service.spec.ts (11 tests) 31ms
[Nest] 27640 - 09/27/2026, 8:46:13 PM LOG [UserService] Provisioning app_user for Keycloak user kc-sub-1 (ann@example.com)
[Nest] 27640 - 09/27/2026, 8:46:13 PM LOG [UserService] Provisioning app_user for Keycloak user kc-sub-1 (ann@example.com)
✓ src/admin/admin-users.service.spec.ts (7 tests) 25ms
✓ src/users/users.service.spec.ts (6 tests) 44ms
✓ src/explore/explore.service.spec.ts (6 tests) 26ms
[Nest] 416 - 09/27/2026, 8:46:13 PM ERROR [AiBackendService] AI backend /trip-plan call failed: Request failed with status code 500
AxiosError: Request failed with status code 500
    at C:\Users\VICTUS\Desktop\SmartJourney-SmartTourismProject\backend/src/chat/ai-backend.service.spec.ts:37:28
    at file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:40
    at runWithSuite (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2258:8)
    at Object.collect (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:10)
    at Object.collect (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1893:54)
    at processTicksAndRejections (node:internal/process/task_queues:103:5)
    at file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2462:26
    at collectTests (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2433:3)
    at startTests (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:3280:17)
    at startTests (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:3280:17)

(.venv) PS C:\Users\VICTUS\Desktop\SmartJourney-SmartTourismProject\backend> npm test
at startTests (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:3280:17)
at file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/vitest/dist/chunks/base.B60p18PE.js:90:5
[Nest] 416 - 09/27/2026, 8:46:13 PM ERROR [AiBackendService] AI backend /trip-plan call failed: timeout of 120000ms exceeded
AxiosError: timeout of 120000ms exceeded
    at C:\Users\VICTUS\Desktop\SmartJourney-SmartTourismProject\backend/src/chat/ai-backend.service.spec.ts:38:19
    at file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:40
    at runWithSuite (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2258:8)
    at Object.collect (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:10)
    at Object.collect (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1893:54)
    at processTicksAndRejections (node:internal/process/task_queues:103:5)
    at file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2462:26
    at collectTests (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2433:3)
    at startTests (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:3280:17)
    at startTests (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:3280:17)
    at file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/vitest/dist/chunks/base.B60p18PE.js:90:5
[Nest] 416 - 09/27/2026, 8:46:13 PM ERROR [AiBackendService] AI backend /trip-plan call failed: connect ECONNREFUSED
AxiosError: connect ECONNREFUSED
    at C:\Users\VICTUS\Desktop\SmartJourney-SmartTourismProject\backend/src/chat/ai-backend.service.spec.ts:39:25
    at file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:40
    at runWithSuite (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2258:8)
    at Object.collect (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1889:10)
    at Object.collect (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:1893:54)
    at processTicksAndRejections (node:internal/process/task_queues:103:5)
    at file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2462:26
    at collectTests (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:2433:3)
    at startTests (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:3280:17)
    at startTests (file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/@vitest/runner/dist/chunk-artifact.js:3280:17)
    at file:///C:/Users/VICTUS/Desktop/SmartJourney-SmartTourismProject/backend/node_modules/vitest/dist/chunks/base.B60p18PE.js:90:5
✓ src/chat/ai-backend.service.spec.ts (5 tests) 28ms
✓ src/auth/jwt.strategy.spec.ts (2 tests) 22ms
✓ src/auth/keycloak-token.spec.ts (2 tests) 7ms
✓ src/chat/chat.service.spec.ts (9 tests) 23ms
✓ src/app.controller.spec.ts (1 test) 419ms
  ✓ should return "Hello World!" 416ms
✓ src/auth/roles.guard.spec.ts (4 tests) 7ms
✓ src/auth/jwt-auth.guard.spec.ts (2 tests) 9ms

Test Files 14 passed (14)
Tests 69 passed (69)
Start at 20:46:10
Duration 4.13s (transform 1.97s, setup 0ms, import 23.53s, tests 709ms, environment 4ms)
```

Figure 19: Backend unit test run (npm test), captured in three parts from start to finish: 14 spec files and 69 tests passing, 0 failed.

5. Risks, Dependencies, Assumptions, and Constraints

Risk	Mitigation	Contingency
Groq free-tier token-per-minute ceiling throttles the LLM failover chain	Gemini is primary, Groq is failover only; accepted as a genuine capacity limit, not a bug	Deterministic fallback plan, never a server error
Nominatim (1 req/sec) / OpenRouteService (500/day) quota exhaustion	Results are cached; calls are batched where possible	Degrade to cached/approximate geocoding rather than fail the request
The AI backend has no authentication of its own	NestJS is the sole trust boundary; the AI backend must be network-isolated from anything but the backend in every deployed environment	If ever exposed directly, treat as a security incident, not a bug report
The raw-SQL migration runner can fail silently	RUNNING.md documents checking its exit code explicitly	Re-run against a known-good snapshot; restore the last verified backup
Keycloak realm-import drift between the export file and a running instance	The realm export is checked into the repo and re-imported on environment setup	Re-export from a known-good instance, diff, re-import
No staging environment	The local Docker Compose stack is the closest equivalent	Manual verification against production-like data before release
LLM-generated output is non-deterministic and may vary between executions. Therefore, exact textual matching is not used as the primary correctness criterion; validation is instead based on structural validity, budget compliance, policy compliance, and other predefined functional constraints.	Test oracles never assert exact text — only schema validity, budget compliance, and plan source	A borderline case is treated as a data-coverage gap, not chased as a flaky test
Load baseline covers one route only (/listings, local, 2026-09-27)	Documented as a partial baseline, not a capacity claim	Extend the baseline to other routes and a deployed environment before any capacity claim is made
A follow-up turn crashes when the planner fails	Open defect, reproduced by golden scenario 6: orchestrator.py's _verify_node validates a missing planner output when an earlier itinerary exists	Guard _verify_node on planner_output itself, so the request degrades to the fallback plan instead of raising

6. References

- [1] Vitest, “Vitest documentation.” [Online]. Available: <https://vitest.dev>
- [2] pytest, “pytest documentation.” [Online]. Available: <https://docs.pytest.org>
- [3] Microsoft, “Playwright documentation.” [Online]. Available: <https://playwright.dev>

[4] Testing Library, “Testing Library documentation.” [Online]. Available: <https://testing-library.com>

[5] NestJS, “Testing,” NestJS Documentation. [Online]. Available: <https://docs.nestjs.com/fundamentals/testing>

[6] Prisma, “Prisma documentation.” [Online]. Available: <https://www.prisma.io/docs>

[7] Keycloak, “Keycloak documentation.” [Online]. Available: <https://www.keycloak.org/documentation>

[8] SmartJourney, “Project master plan: 12 golden scenarios and their results,” internal document, ai-backend/docs/master_plan/PROJECT_MASTER_PLAN.md, Sec. 6.

[9] SmartJourney, “Determinism and validation: LLM output validation,” internal document, ai-backend/docs/master_plan/DETERMINISM_AND_VALIDATION.md.

- Internal: REPO_STATUS.md, RUNNING.md - per-repo state and the local stack startup order

Appendix A — Manual Test Log Template

Date	Tester	Test case	Executed	Pass	Fail	Pass %	Fail %	Comments
2026-09-27	Shaluka	Manual evidence checks: Prisma Studio, DevTools page load, autocannon load run, Keycloak realm roles (Figures 1, 6, 7, 9)	4	4	0	100%	0%	First page-load and load-test baselines recorded (Section 3.1.4, Section 3.1.5)
2026-09-27	Thisuri	Golden scenarios: python scripts/e2e_check.py against the full local stack (Figure 10)	12	10	2	83.3%	16.7%	First attempt 8/12 from a harness bug (fixed). Scenario 5: day-2 cost not lower. Scenario 6: unhandled exception, open defect (Section 4.1)
2026-09-27	Shaluka	CI workflows on first push to GitHub, all 3 repos (Figures 11–13)	3	3	0	100%	0%	Green only after configuration fixes; all three failed on their first push (Section 3.1.8)

Appendix B — Screenshot Checklist

#	Section	Command / where to look	What to capture
1	3.1.1	<code>npx prisma studio</code> (in backend/)	Table list with a few rows of real data
2	3.1.2	<code>npm test</code> (in backend/)	"69 passed" summary
3	3.1.2	<code>python -m pytest -m "not external" -q</code> (in ai-backend/, venv active)	"544 passed" summary
4	3.1.3	<code>npm test</code> (in frontend-web/)	"18 passed" summary
5	3.1.3	<code>npx playwright show-report</code> (after <code>npm run test:e2e</code>)	HTML report: 1 passed, 3 skipped
6	3.1.4	Chrome DevTools → Network tab, /home page	Request waterfall + total load time
7	3.1.5	<code>npx autocannon -c 20 -d 30 http://localhost:3001/listings</code>	Requests/sec + latency table
8	3.1.6	<code>npm run test:e2e</code> (in backend/)	List of 6 passing test names
9	3.1.6	Keycloak admin console → Realm roles	"traveler" / "admin" roles listed
10	3.1.7	<code>python scripts/e2e_check.py</code> (in ai-backend/, full stack up)	12-scenario scorecard and the final Result line
11–13	3.1.8	GitHub → Actions tab, each repo	Green checkmark + expanded steps
14–15	4.1	<code>npm run test:cov</code> (backend, then frontend-web)	Per-file coverage table, both repos